FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS

CSE3433 SOFTWARE ARCHITECTURE

SEMESTER II SESSION 25/26

PROJECT TITLE:
GROUP 14
RECRUITX - EVENT-DRIVEN JOB RECRUITMENT & APPLICATION PROCESSING SYSTEM

PROJECT REPORT

PREPARED FOR:
DR. MOHAMAD NOR BIN HASSAN

PREPARED BY:

NAME	MATRIC NO.
NUR ALIN HAYANI BINTI AHMAD SYUKRI	S75034
AINAA NADHIRAH BINTI ASMADI	S75246
MARYAM MUSFIRAH BINTI ARIFF HAZIZAN	S75922

TABLE OF CONTENT

1.0 Introduction

RecruitX is a web-based recruitment system developed to improve and automate the hiring process. Traditional recruitment methods often involve manual resume reviews, candidate tracking, and communication, which can be time-consuming and inefficient. RecruitX addresses these challenges by providing features such as job posting management, application submission, resume screening, interview scheduling, and application status tracking.

The system adopts an Event-Driven Architecture (EDA), where user actions trigger events that automatically activate related processes. This approach improves efficiency, reduces manual work, and enables faster communication between candidates and recruiters.

The system supports three main users:

- Candidate – applies for jobs and tracks application status
- Recruiter / HR Staff – manages job postings and candidate progress
- System Services – handle automation such as screening, notifications, and analytics

2.0 Problem Description

The current recruitment is heavily manual and slow, which causes several problems for the organization:

- **Slow hiring process**: HR teams must read through every resume and contact candidates one by one. This takes a lot of time and effort, making the whole process move very slowly
- **No immediate action**: When a candidate submits an application, nothing happens automatically. The application just sits there until an HR staff member manually opens it, leading to long delays in shortlisting
- **Poor communication**: Candidates often wait a long time for updates.
- **Risk of mistakes**: Handling everything by hand makes it easy for humans to make errors.
- **Difficulty handling many applicants**: If a job gets a large number of applications, the manual system becomes overwhelmed. The team cannot work fast enough to keep up with the workload.

3.0 System Requirement

3.1 Functional Requirements

FR01 - The system shall allow all the recruiters to create and publish the job listings which will trigger a JobPosted event to update the public job board automatically.

FR02 - The system shall, when submission of an application, trigger an ApplicationReceived event to provide the candidate with an intermediate automated confirmation.

FR03 - The system shall trigger an ApplicationStatusChanged event when a recruiter updates all the candidate progress, automatically sending an update to the candidate to eliminate the communication delays.

FR04 - The system shall automatically screen the incoming resumes based on pre-defined keywords like skills or year of experience to assist HR in narrowing down to the applicant pool without manual effort.

FR05 - The system shall provide a centralized interface for recruiters to move the candidate through stages like shortlisted or interviewed, ensuring no qualified candidate is overlooked.

FR06 - The system shall allow the recruiters to initiate the InterviewRequested event which is automatically coordinated with the candidate's availability and sends the calendar invites.

FR07 - The system shall provide a dashboard that monitors the "hiring funnel" in real-time, reducing all the human error in data entry or tracking.

3.2 Non-Functional Requirements

NFR-01- Availability; The system shall be able to maintain an uptime of 99.5%, ensuring that recruiters and candidates can access the platform at any time including during peak hiring periods.

NFR-02 - Scalability: The system shall be able to handle a sudden surge of up to 1,000 concurrent event messages without a decrease in performance or message loss.

NFR-03 - Reliability: The system shall ensure guaranteed message delivery if an event fails to reach a consumer, the event bus must retry the delivery until successful.

NFR-04 - Performance: The system shall ensure that event triggers occur within 3 seconds of the candidate clicking the “Submit” button.

NFR-05 - Security: The system shall encrypt all sensitive candidate data at rest while it is stored in the database and while it is being send over the network

NFR-06 - Maintainability: The system architecture shall be modularly decoupled, allowing new event consumers to be added without requiring a reboot of the main application.

4.0 Architecture Design

RecruitX uses an Event-Driven Architecture (EDA) with a Publish/Subscribe model, where components communicate by producing and consuming events instead of directly interacting *(GeeksforGeeks, 2024)* .

4.1 Event Producers

- Candidate Portal
    - Publishes events such as ApplicationReceived when a candidate submits a job application
- Recruiter / HR Portal
    - Publishes events like JobPosted, ApplicationStatusChanged, and InterviewRequested when recruiters perform actions

4.2 Event Bus

- Simulated RabbitMQ Event Dispatcher

4.3 Event Consumers

- Application Management Service
    - Stores application data when an ApplicationReceived event occurs
- Resume Screening Service
    - Automatically evaluates resumes based on predefined criteria
- Notification Service
    - Sends alerts when events like application submission or status updates occur
- Interview Scheduling Service
    - Handles interview requests and sends calendar invitations
- Analytics Service
    - Collects data from events to generate reports and insights

5.0 Architecture Diagram

5.1 System Context Diagram

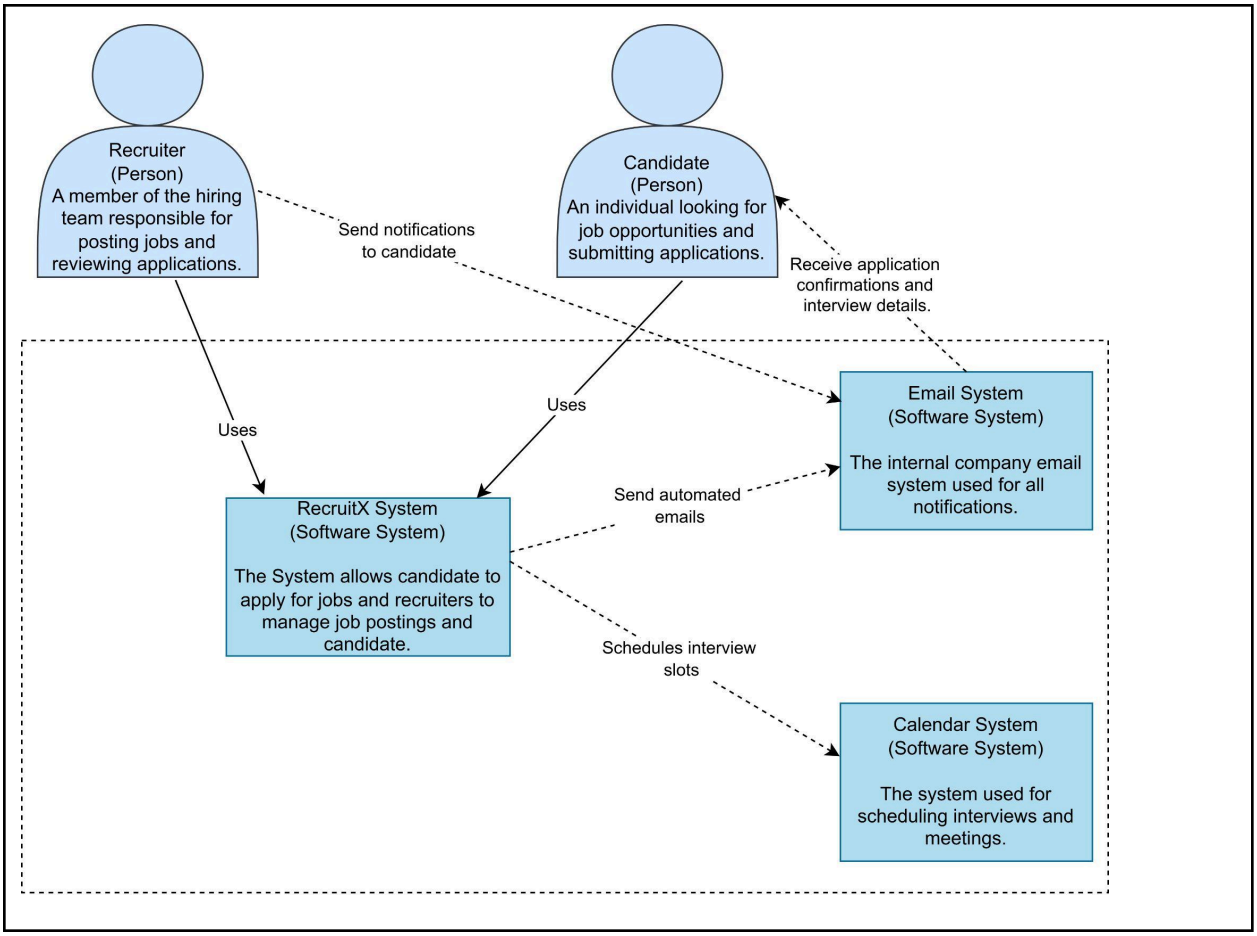


Figure 5.1 System Context Diagram RecruitX System

Figure 5.1 shows the System Context Diagram of RecruitX. The diagram illustrates the interaction between candidates, recruiters, the RecruitX system, and external services.

Candidates use the system to apply for jobs and track application progress, while recruiters manage job postings and candidate applications. RecruitX also connects with email and calendar services to support notifications and interview scheduling.

Overall, the diagram shows how RecruitX serves as the central platform that coordinates recruitment activities and communication between users and external systems.

5.2 Component / Service Architecture Diagram

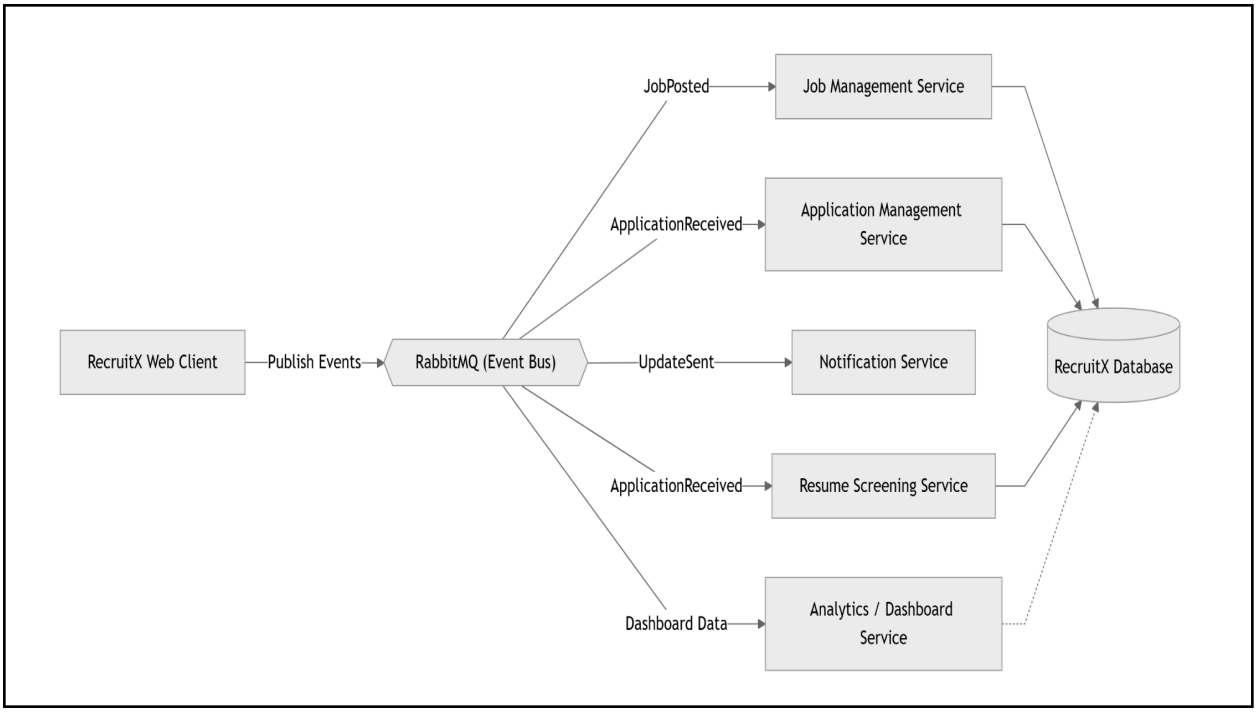


Figure 5.2 Component / Service Architecture Diagram for RecruitX system

This is the component diagram which depicts how the key services work and interact within the RecruitX. In the event-driven architecture, events are propagated through the Event Bus (RabbitMQ), which in turn propagates them to the relevant services like Application processing service, Notification service, and interview scheduling service.

5.3 Interaction Diagram

Candidate Application Interaction Flow

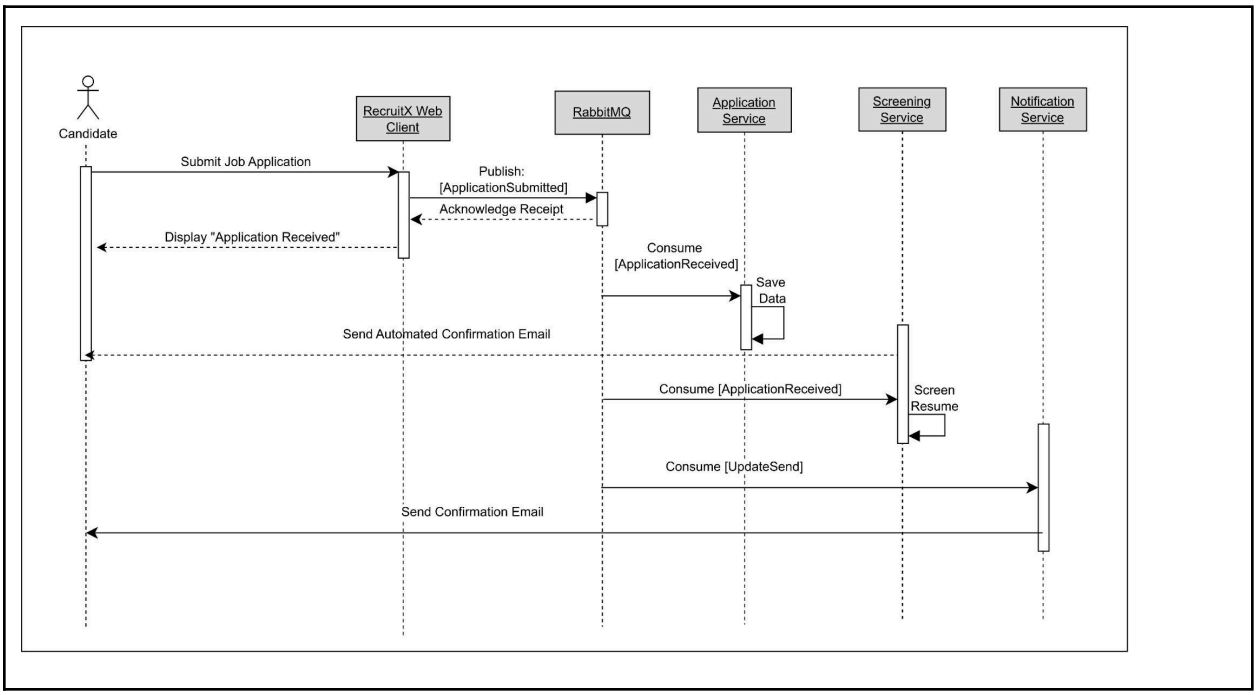


Figure 5.3.1 Candidate Application Interaction Flow for RecruitX system

Figure 5.3.1 illustrates the candidate application interaction flow in RecruitX. When a candidate submits a job application, the system publishes an event that is processed asynchronously by multiple services.

The Application Service stores the application data, the Screening Service evaluates the resume, and the Notification Service sends a confirmation message to the candidate. This event-driven approach improves system efficiency, scalability, and responsiveness by allowing services to operate independently.

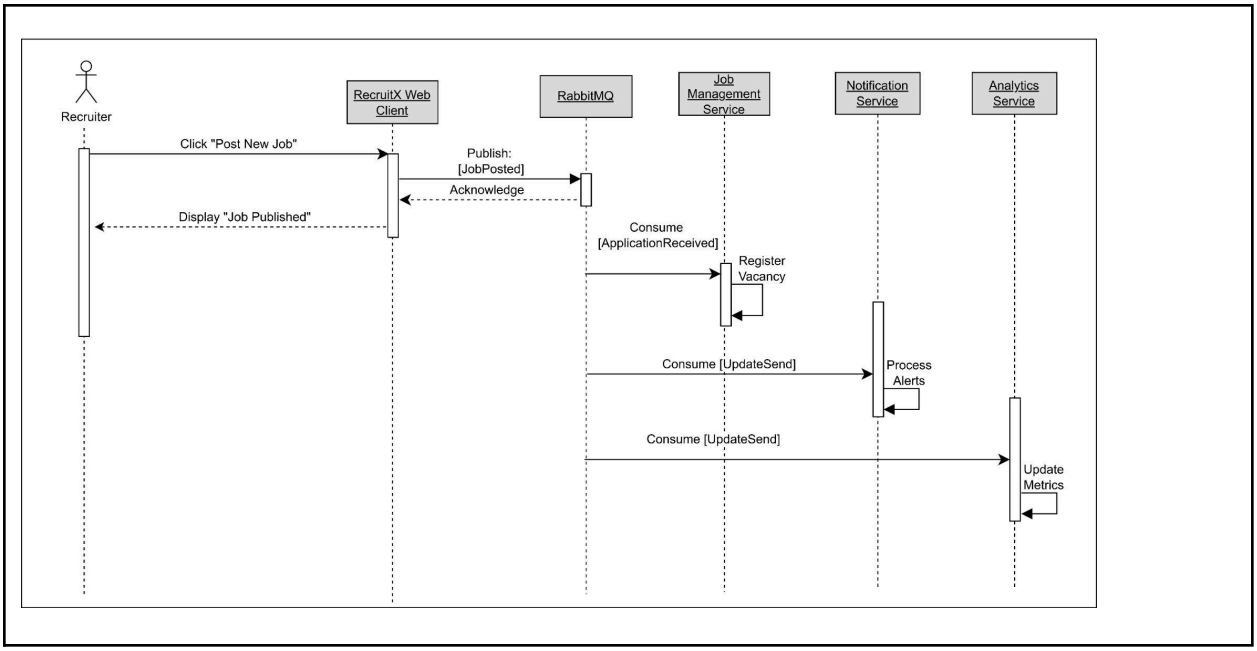


Figure 5.3.2 Recruiter Interaction Flow for RecruitX system

Figure 5.3.2 illustrates the recruiter interaction flow in RecruitX. When a recruiter posts a new job vacancy, the system publishes a JobPosted event and immediately provides confirmation to the recruiter.

The event is then processed asynchronously by multiple services. The Job Management Service registers the vacancy, the Notification Service handles candidate alerts, and the Analytics Service updates recruitment metrics. This event-driven workflow improves system efficiency and allows services to operate independently while maintaining consistent recruitment data.

5.4 Deployment Diagram

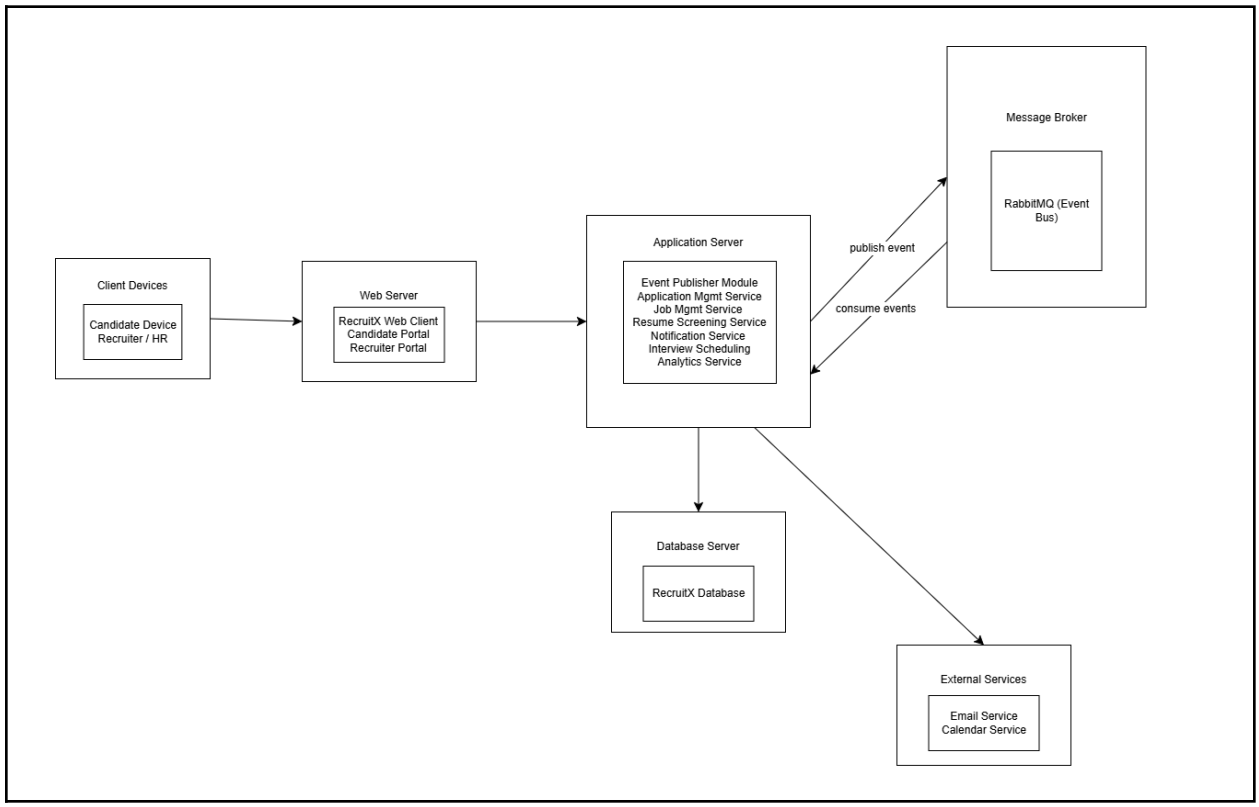


Figure 5.4 : Deployment Diagram for RecruitX system

Figure 5.4 illustrates the deployment architecture of RecruitX. Users access the system through client devices connected to the web server, which hosts the Candidate and Recruiter portals.

The web server communicates with the application server, where core services such as application management, resume screening, notification, interview scheduling, and analytics are executed. The application server also interacts with the RecruitX database and publishes events through RabbitMQ for asynchronous processing.

This deployment structure supports efficient communication between system components and enables scalable event-driven operations.

6.0 Technology Stack

Component	Technology	Description
Frontend	JSP, HTML, CSS	Build dynamic web views for the Candidate and Recruiter portals, handling client-side UI rendering
Backend	Java Servlet	Powers server-side request processing and maps user interaction logic.
Architecture	Event Driven Architecture	Implements a Publish/Subscribe pattern to asynchronously decouple system modules.
Database	MySQL	Manage relational data following the decentralized “Database per Service” pattern
IDE	NetBeans	The primary Integrated Development Environment used for compiling and writing the Java Web codebase.
Application Server	Apache Tomcat	Act as the standard servlet container to deploy, execute, and host the RecruitX web modules
Event Bus	Simulated RabbitMQ	Simulates message queues and asynchronous event routing between services to ensure the loose coupling
Version Control	Github	The central repository platform for team code collaboration, version history and branching.
Database Tool	phpMyAdmin	The web interface used to administer data schemas and directly verify microservice database updates.

7.0 Prototype Implementation

7.1 Candidate Portal

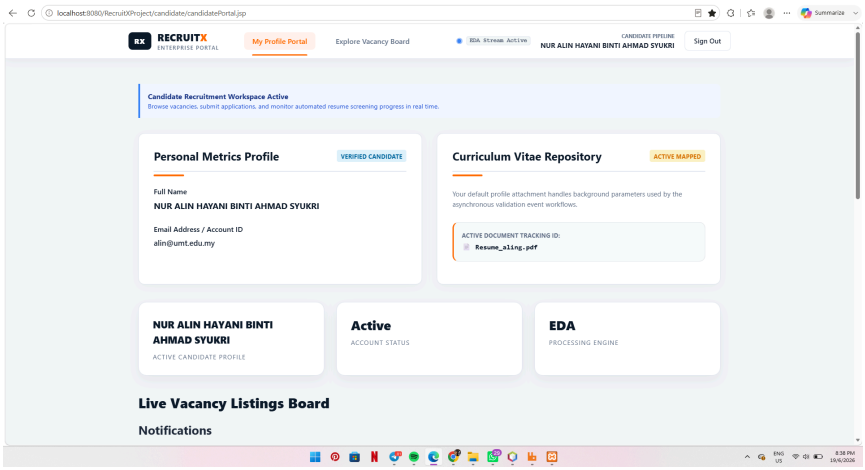


Figure 7.1.1 Candidate recruitment workflow in RecruitX

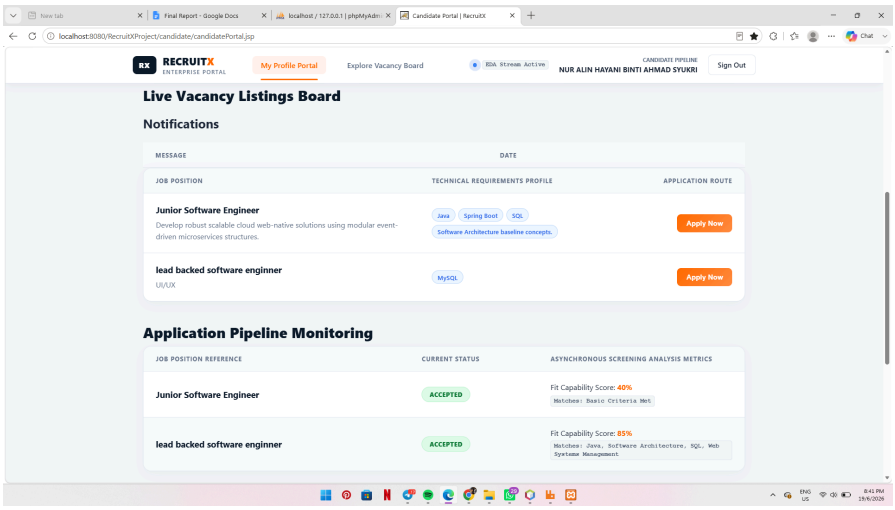


Figure 7.1.2 Candidate recruitment workflow in RecruitX

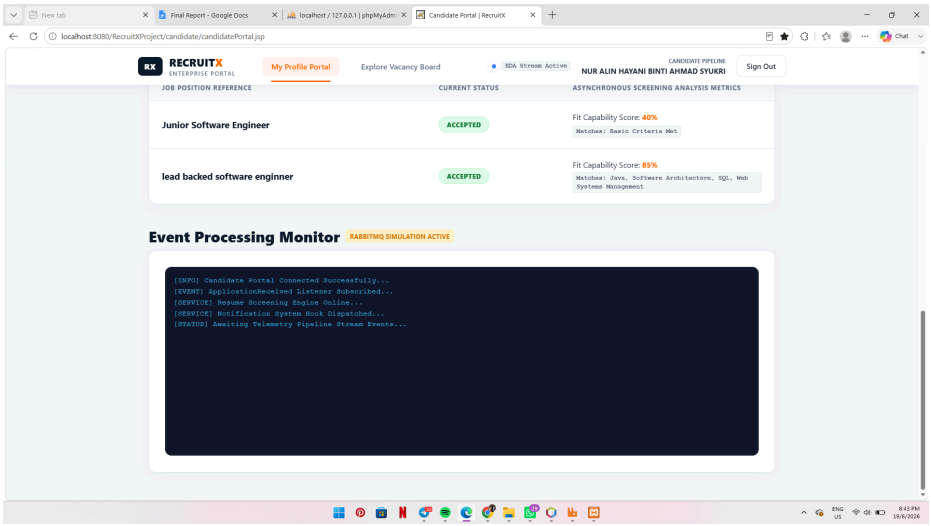


Figure 7.1.3 Candidate recruitment workflow in RecruitX

Figure 7.1.1 - 7.1.3 shows the candidate recruitment workflow in RecruitX. The portal allows candidates to view available job vacancies, submit applications, and monitor their application progress. After an application is submitted, the system automatically processes the application through the Event-Driven Architecture (EDA) workflow, performs resume screening, calculates a fit score, and updates the application status. Candidates can also receive notifications and track recruitment progress in real time through the application monitoring dashboard.

7.2 Recruiter Dashboard

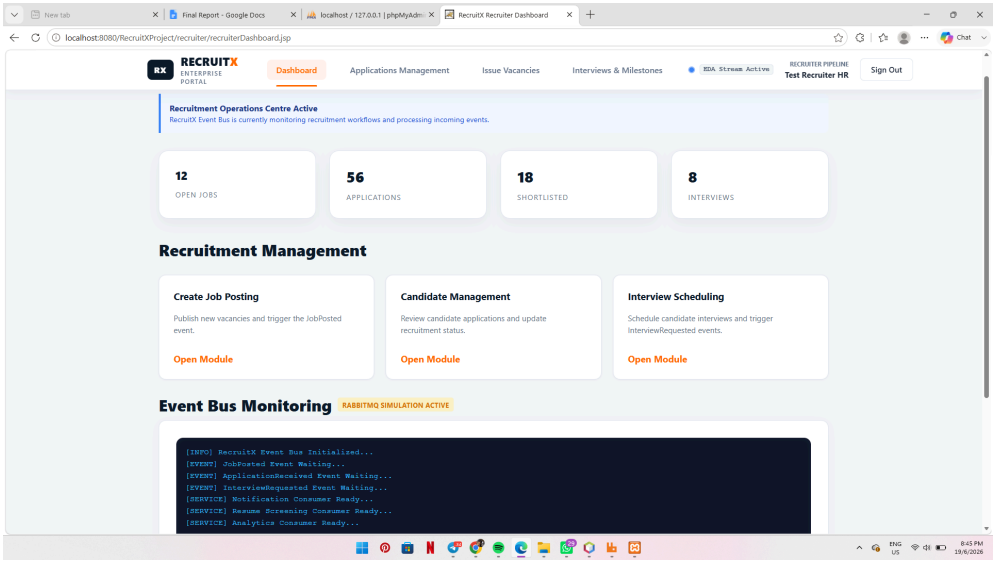


Figure 7.2 Recruiter Dashboard of RecruitX

Figure 7.2 shows the Recruiter Dashboard of RecruitX. This interface allows recruiters to manage recruitment activities, including creating job postings, reviewing candidate applications, updating application statuses, and scheduling interviews. The dashboard also provides recruitment statistics and an Event Bus Monitoring section that displays the status of event-driven services. Through this portal, recruiters can efficiently manage the hiring process while supporting the automated workflows implemented using Event-Driven Architecture (EDA).

7.3 Candidate Application Management

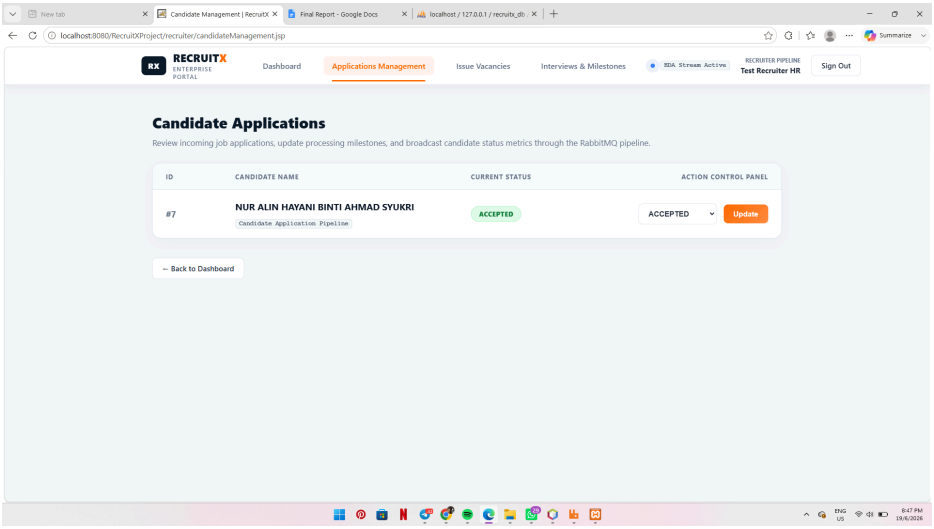


Figure 7.3 Candidate Applications Management

Figure 7.3 shows the Candidate Applications Management module used by recruiters to review submitted applications and manage candidate progress. Recruiters can view application details, monitor the current status, and update recruitment outcomes such as Accepted, Shortlisted, or Rejected. Status updates are processed through the event-driven workflow, ensuring that related services and candidate notifications are updated automatically.

7.4 Create Job Posting

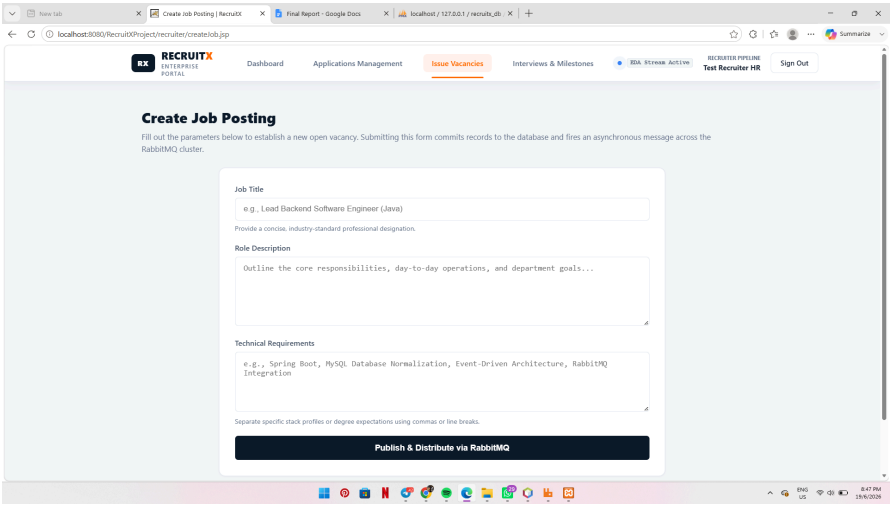


Figure 7.4 Create Job Posting

Figure 7.4 shows the Create Job Posting module used by recruiters to publish new vacancies. Recruiters can enter the job title, role description, and technical requirements before submitting the vacancy. Once submitted, the job information is stored in the database and a **JobPosted** event is published through the event-driven architecture, allowing other services such as notification and analytics to process the information automatically.

7.5 Interview Scheduling

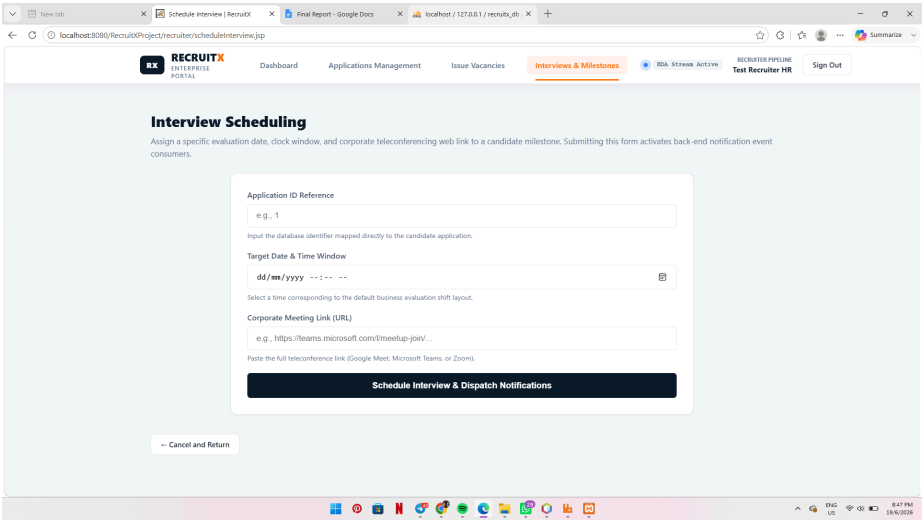


Figure 7.5 Interview Scheduling module

Figure 7.5 shows the Interview Scheduling module, which allows recruiters to schedule interviews for shortlisted candidates. Recruiters can enter the application ID, interview date and time, and meeting link. After submission, the interview details are stored in the database and an **InterviewScheduled** event is triggered to automatically notify the candidate through the system.

7.6 Database Integration

7.6.1 Users table

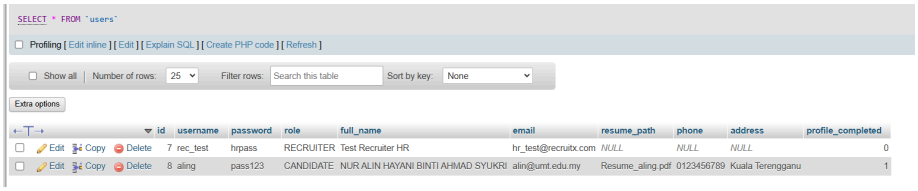


Figure 7.6.1 Users table

Figure 7.6.1 shows the **users** table, which stores account information for both candidates and recruiters. The table contains user credentials, personal details, role assignments, contact information, and resume references used throughout the recruitment process.

7.6.2 Screening Results table

Figure 7.6.2 Screening Results

Figure 7.6.2 shows the **screening_results** table, which stores the results generated by the automated resume screening service. The table records fit scores, keyword matches, and screening outcomes used to support candidate evaluation.

7.6.3 Jobs table

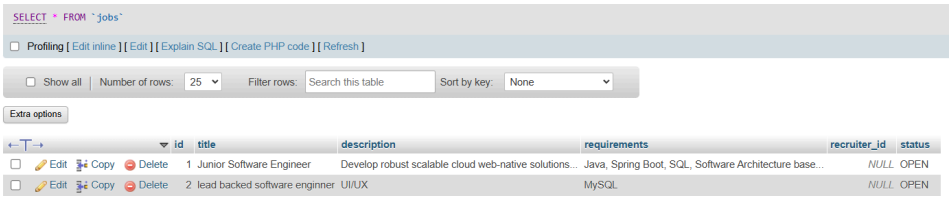


Figure 7.6.3 Jobs table

Figure 7.6.3 shows the **jobs** table, which stores job vacancies created by recruiters. Each record contains the job title, description, technical requirements, recruiter information, and current vacancy status.

7.6.4 Interviews table

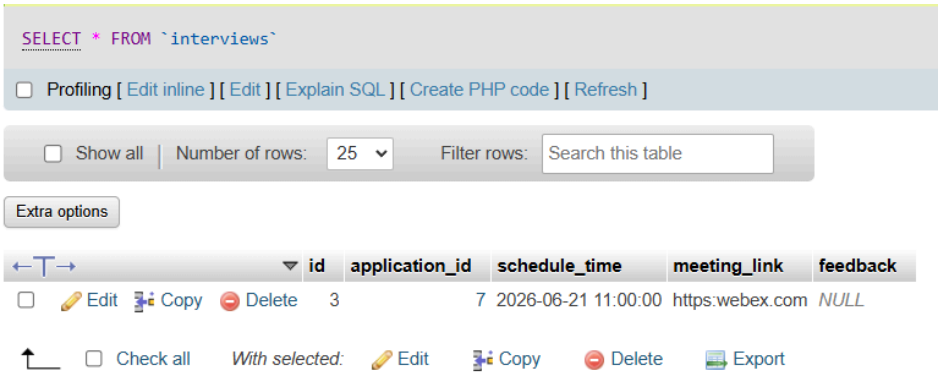
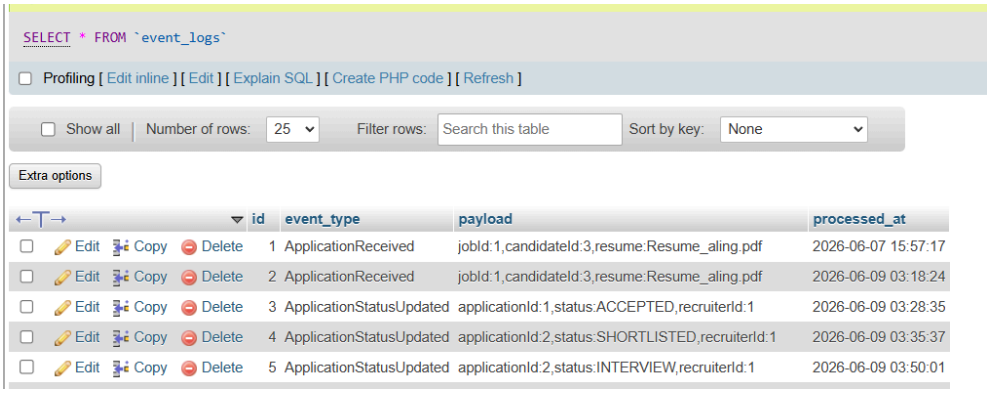


Figure 7.6.4 Interviews table

Figure 7.6.4 shows the **interviews** table, which stores interview schedules created by recruiters. The table contains the application reference, interview date and time, meeting link, and interview feedback information.

7.6.5 Event Logs table



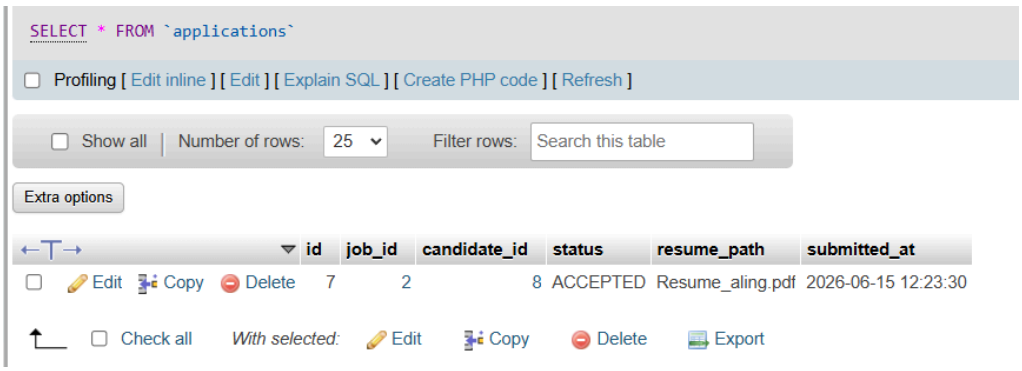
```
SELECT * FROM `event_logs`
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
Extra options
```

	id	event_type	payload	processed_at
Edit Copy Delete	1	ApplicationReceived	jobId:1,candidateId:3,resume:Resume_aling.pdf	2026-06-07 15:57:17
Edit Copy Delete	2	ApplicationReceived	jobId:1,candidateId:3,resume:Resume_aling.pdf	2026-06-09 03:18:24
Edit Copy Delete	3	ApplicationStatusUpdated	applicationId:1,status:ACCEPTED,recruiterId:1	2026-06-09 03:28:35
Edit Copy Delete	4	ApplicationStatusUpdated	applicationId:2,status:SHORTLISTED,recruiterId:1	2026-06-09 03:35:37
Edit Copy Delete	5	ApplicationStatusUpdated	applicationId:2,status:INTERVIEW,recruiterId:1	2026-06-09 03:50:01

Figure 7.6.5 Event Logs Table

Figure 7.6.5 shows the **event_logs** table, which records all events processed by the Event-Driven Architecture. The table stores event types, payload information, and processing timestamps to support monitoring and system auditing.

7.6.6 Application table



```
SELECT * FROM `applications`
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
```

	id	job_id	candidate_id	status	resume_path	submitted_at
Edit Copy Delete	7	2	8	ACCEPTED	Resume_aling.pdf	2026-06-15 12:23:30

Check all *With selected:* Edit Copy Delete Export

Figure 7.6.6 Application Table

Figure 7.6.6 shows the **applications** table, which records job applications submitted by candidates. The table links candidates to job postings and stores application status, resume information, and submission timestamps.

8.0 Architecture Evaluation

The implementation successfully demonstrates Event-Driven Architecture principles

Strengths	**Description**
Loose Coupling	Consumers operate independently
Scalability	Additional consumers can be added easily
Asynchronous Processing	Events trigger background processing without blocking users
Traceability	All events are stored in the **event_logs** table

Limitations	**Description**
Simulated RabbitMQ	A real RabbitMQ broker was not deployed
Basic Resume Screening	Keyword matching is currently rule-based
Limited Analytics	Analytics functionality currently records event activities only

9.0 Team Contributions

Name	Role	Architectural Design & Modeling Contribution	Prototype & Implementation Contribution
Nur Alin Hayani binti Ahmad Syukri	System Architect & Designer	Responsible for UML Architecture Diagrams, Justify the Architectural Drivers	Designed frontend UI mockups and client-side view routers for the portals.
Ainaa Nadhirah binti Asmadi	Backend & Integration Lead	Backend service development, database management and API implementation.	Developed independent backend services and implemented Database Managers.
Maryam Musfirah binti Ariff Hazizan	Requirements Analyst	Analyzing the problem statement, defining system objectives, preparing FR, and identifying architectural drivers.	Wrote backend testing scripts, validated message consumer logs, and compiled final evaluation benchmarks.

10.0 Conclusion and Reflection

10.1 Conclusion

RecruitX successfully demonstrates the implementation of Event-Driven Architecture within a recruitment management system. The project automates recruitment workflows including application submission, resume screening, status updates, notifications, and interview scheduling.

The system achieved its primary objectives by reducing manual intervention and improving recruitment efficiency through asynchronous event processing. Database evidence confirms successful interaction between the event bus simulation and backend services.

Future enhancements include integrating a real RabbitMQ message broker, implementing AI-based resume screening, email delivery services, and advanced analytics dashboards.

10.2 Reflection

Throughout 10 weeks of this project, our team overcome some critical technical and design challenges and gain invaluable experience insight which is:

- Shift to an Asynchronous Model: We transitioned from the traditional monolithics synchronous web structure to the event driven flow require the major shift to logic. Our group learned how to design the system using consistency, discovering how to provide immediate user facing confirmation such as instant UI client receipt acknowledgement while executing the complex background business logic such as text extraction and multi service event consumption which is running in parallel.

- Data Isolation and Fault Tolerance: Designing under the database pattern highlighted the practical value of the data ownership. We realized that by drawing the strict service boundaries, a failure in a secondary service such as a temporary crash in the Analytics Service which would remain isolated and never break the core platform operations like candidate job application and database persistence.